

基于 MiniOB 的数据库管理系统设计与实现

数据库系统原理课程设计实验报告

开发平台：MiniOB（OceanBase 开源教学版）

开发语言：C/C++，Lex/Yacc

报告版本：完整版草案

封面信息

项目	内容
课程设计名称	数据库系统原理课程设计
学院	计算机学院
班级	TODO
指导教师	TODO
完成日期	TODO
课程设计内容	基于 MiniOB 实现 SQL 解析、语义绑定、查询执行、索引维护、子查询、NULL 语义与大规模读写验证等数据库内核功能

小组成员与分工

成员	学号	主要分工
成员 A	TODO	TODO：基础功能、DDL/DML、测试框架等
成员 B	TODO	TODO：join、group-by、multi-index、complex-sub-query、big-write 等
成员 C	TODO	TODO：null、simple-sub-query、update-select、big-query、报告整理等

截图占位：封面最终 Word 版按指导书模板补充班级、姓名、学号、指导教师与成绩栏。

1. 任务分析

1.1 实验目的

本课程设计基于 MiniOB 教学数据库系统，要求在已有数据库内核框架上补全 SQL 功能，并通过编译、运行和测试验证功能正确性。实验的核心目标不是单纯补语法，而是理解一条 SQL 从文本输入到最终访问存储引擎的完整路径。

MiniOB 的主执行链可以概括为：

SQL 文本

```
-> 词法/语法分析 (lex_sql.1 / yacc_sql.y)
-> ParsedSqlNode
-> Stmt 语义绑定 (表、字段、类型、约束)
-> LogicalPlanGenerator
-> PhysicalPlanGenerator
-> PhysicalOperator 火山模型执行
-> Trx / Table / HeapTableEngine / B+Tree Index
```

本实验围绕这条链路完成多类功能：

- DDL：表删除、索引创建、唯一索引等。
- DML：插入、删除、更新、带子查询的更新。
- 查询：JOIN、聚合、分组、HAVING、排序、函数、LIKE。
- 类型与语义：DATE、NULL、表达式求值。
- 子查询：简单子查询、复杂关联子查询。
- 稳健性：大规模写入与大规模查询压力测试。

1.2 完成功能总览

本组共完成 18 项功能，其中包含困难题 `complex-sub-query`、`big-write`、`big-query`。功能列表如下。

序号	功能	难度	分值	类别	完成情况
1	basic	简单	2	基线能力	完成
2	drop-table	简单	2	DDL	完成
3	update	简单	2	DML	完成
4	date	简单	2	类型系统	完成
5	aggregation-func	简单	2	查询	完成
6	like	简单	2	查询	完成
7	join-tables	中等	2	查询	完成
8	function / expression	中等	2	表达式	完成
9	order-by	中等	2	查询	完成
10	group-by	中等	4	查询	完成
11	multi-index	中等	4	索引	完成
12	unique	中等	2	索引	完成
13	simple-sub-query	中等	4	子查询	完成
14	null	中等	3	SQL 语义	完成
15	update-select	中等	4	DML + 子查询	完成
16	complex-sub-query	困难	5	子查询	完成
17	big-write	困难	5	稳健性	完成
18	big-query	困难	5	稳健性	完成

合计约 54 分，具体分值以课程指导书与验收脚本为准。困难题部分至少完成 `complex-sub-query`，并额外完成 `big-write` 与 `big-query` 两类稳健性题目。

1.3 总体设计路线

本组实现遵循 MiniOB 原有分层结构，尽量避免旁路实现。主要设计原则如下：

- 语法扩展只负责构造 AST 或表达式节点，不在 parser 层直接执行语义。
- Stmt 层负责表名、字段名、类型、约束和作用域绑定。
- 查询类功能统一进入逻辑计划与物理计划，不绕过火山模型算子。
- 子查询统一抽象为表达式节点，使 SELECT WHERE、UPDATE SET 等位置可以复用。
- DML 修改记录时同步维护索引，失败时尽量回滚，保证堆表与索引一致。
- 压力测试使用固定随机种子，保证失败可复现。

1.4 未实现功能

本版本未实现以下扩展题目：

- text（TEXT 类型）
- alias（表/列别名）
- create-view
- create-table-select
- update-mvcc
- big-order-by
- dblink 系列优化题（hash join、optimizer、sort）
- vectorized 向量化执行

报告后续只对已完成的 18 项功能展开说明，不将未实现题目计入完成情况。

2. 开发环境配置

2.1 macOS 本机开发环境

部分开发与测试在 macOS 本机完成。MiniOB 使用 CMake、Flex、Bison、C++ 编译器和第三方依赖库。本机开发流程如下：

1. 安装 Xcode Command Line Tools。
2. 使用 Homebrew 安装 `cmake`、`flex`、`bison`。
3. 确认 `bison --version` 为 3.x。macOS 系统默认 Bison 2.3 无法解析 MiniOB 的 `yacc_sql.y`。
4. 初始化子模块：`git submodule update --init --recursive`。
5. 初始化依赖：`bash build.sh init`。
6. 编译 observer：`bash build.sh debug --make -j8`。
7. 启动 CLI：`build_debug/bin/observer -f etc/observer.ini -P cli`。

常见问题：

问题	原因	处理方式
<code>yacc_sql.y</code> 语法错误	Bison 版本过低	安装 Bison 3.x 并调整 PATH
重复测试出现建表失败	数据目录残留	清空 <code>build_debug/bin/miniob/db/sys/</code>
第三方依赖初始化失败	子模块状态异常	恢复子模块后单独编译依赖

截图占位：macOS 编译成功截图、observer CLI 启动截图。

2.2 Windows 开发环境

Windows 环境建议采用 Docker Desktop 或 WSL2，避免直接在 Windows 原命令行中处理 Flex/Bison 和 Linux 依赖。

推荐方案：

- 安装 Docker Desktop 并启用 WSL2 backend。
- 使用课程推荐的 MiniOB 开发镜像或等价 Linux 容器。
- 将仓库挂载到容器内，在容器中执行 `cmake`、`bison`、`make`。

- 使用 VS Code Remote Containers 或 CLion 远程工具链连接容器。

参考命令：

```
docker exec miniob-dev bash -lc "cd /miniob && cmake --build build -j4"
```

Windows 本机主要承担代码编辑、报告整理和 Docker 容器控制；最终编译与联调可在统一 Linux 工具链中执行。

截图占位：Windows Docker Desktop 容器运行截图、容器内编译命令截图。

3. 功能实现

3.1 basic

功能目标

`basic` 是 MiniOB 的基线能力，覆盖基础建表、插入、查询、删除、索引创建等操作。它为后续功能提供回归基准。

设计思路

基础功能沿用 MiniOB 原有框架，不进行旁路改造。后续所有功能都必须保证 basic 用例不回归。

实现链路

```
CREATE / INSERT / SELECT / DELETE
-> parser
-> Stmt
-> logical plan
-> physical operator
-> table / trx / record
```

代码锚点

- `src/observer/sql/parser/yacc_sql.y`
- `src/observer/sql/stmt/stmt.cpp`
- `src/observer/sql/optimizer/logical_plan_generator.cpp`
- `src/observer/sql/operator/*`

测试验证

- 官方用例： `official-basic`
- 自定义用例： `custom-basic`
- 综合断言： `comp-001` 至 `comp-004`

3.2 drop-table

功能目标

支持 `DROP TABLE table_name`，删除表定义、数据文件和相关索引资源。删除不存在的表应返回失败，删除后允许同名重建。

设计思路

`DROP TABLE` 属于 DDL，不需要走查询计划和物理算子，而应与 `CREATE TABLE` 类似，经命令执行器进入数据库 catalog 层。这样可以集中管理表对象、元数据和磁盘文件。

实现链路

```
DROP TABLE t
-> yacc_sql.y 生成 SCF_DROP_TABLE
-> Stmt::create_stmt
-> CommandExecutor
-> Db::drop_table
-> 删除 table meta、data file、index file
```

关键代码锚点

- `src/observer/sql/parser/yacc_sql.y`：解析 `drop_table_stmt`。
- `src/observer/sql/stmt/stmt.cpp`：分发 `SCF_DROP_TABLE`。
- `src/observer/sql/executor/command_executor.cpp`：执行 DDL 命令。
- `src/observer/storage/db/db.cpp`：删除表对象与文件。

测试验证

典型 SQL：

```
CREATE TABLE dt(id INT, name CHAR(10));
INSERT INTO dt VALUES (1, 'x');
DROP TABLE dt;
CREATE TABLE dt(id INT);
SELECT * FROM dt;
DROP TABLE dt_not_exist;
```

验证点：已存在表可删除，删除后可重建，不存在表删除失败。

3.3 update

功能目标

支持：

```
UPDATE table_name SET column = value [WHERE condition];
```

包括条件更新、全表更新、更新索引列、更新非索引列、错误表/列/类型处理。

设计思路

`UPDATE` 属于 DML，应复用 `DELETE` 的执行路径：先通过子算子扫描命中记录，再由物理更新算子逐条修改。由于更新可能影响索引，需要在存储引擎层保证堆表记录与 B+ 树索引同步。

实现链路

```
UPDATE t SET c=v WHERE ...
-> UpdateStmt::create
-> TableGet + Predicate + UpdateLogicalOperator
-> UpdatePhysicalOperator::open
-> Trx::update_record
-> HeapTableEngine::update_record_with_trx
-> 删除旧索引项 -> 修改记录 -> 插入新索引项
```

关键代码锚点

- `src/observer/sql/stmt/update_stmt.*`: 校验表、字段、WHERE 条件。
- `src/observer/sql/operator/update_logical_operator.*`: 保存更新目标。
- `src/observer/sql/operator/update_physical_operator.*`: 执行逐行更新。
- `src/observer/storage/table/heap_table_engine.*`: 维护记录与索引一致性。

测试验证

典型场景:

- 单行条件更新。
- 多行条件更新。
- 无 WHERE 全表更新。
- 更新索引列后按索引查询结果正确。
- 表不存在、字段不存在、非法类型赋值失败。

3.4 date

功能目标

支持 DATE 类型，能够插入、比较、排序和建立索引。非法日期需要被拒绝，例如非闰年的 `2017-02-29`。

设计思路

DATE 在 SQL 层表现为日期字符串，在内部存储中可编码为整数。这样比较、排序和索引比较可以复用数值比较逻辑，同时保留日期合法性校验。

实现链路

```
DATE 字面量 / DATE 字段
-> parser 识别 AttrType::DATES
-> Value 保存日期编码
-> DateType 校验年月日合法性
-> ComparisonExpr / Index key 使用统一比较逻辑
```

关键代码锚点

- `src/observer/common/type/date_type.*`
- `src/observer/common/value.*`
- `src/observer/sql/parser/yacc_sql.y`

- `src/observer/storage/index/*`

测试验证

典型场景：

- 合法日期插入成功。
- 非法日期插入失败。
- DATE 字段参与 WHERE 比较。
- DATE 字段建索引后结果与全表扫描一致。

3.5 aggregation-func

功能目标

支持 `COUNT`、`MIN`、`MAX`、`AVG`、`SUM`，并支持 `COUNT(*)`。非法聚合写法应返回失败。

设计思路

聚合函数本质上是对输入多行进行状态累积。MiniOB 中将聚合函数抽象为聚合器，扫描输入 tuple 时更新聚合状态，扫描结束后输出最终值。

实现链路

```
SELECT avg(c), count(*) FROM t
-> parser 生成聚合表达式
-> binder 识别聚合函数参数
-> ScalarGroupByPhysicalOperator
-> Aggregator 累积状态
-> Project 输出最终结果
```

关键代码锚点

- `src/observer/sql/expr/aggregator.*`
- `src/observer/sql/expr/expression.*`
- `src/observer/sql/operator/scalar_group_by_physical_operator.*`
- `src/observer/sql/operator/hash_group_by_physical_operator.*`

测试验证

验证 `COUNT(*)`、`COUNT(col)`、`MIN/MAX/AVG/SUM`，以及不存在列、非法 `MIN(*)` 等错误场景。

3.6 like

功能目标

支持：

```
WHERE col LIKE 'pattern'
```

其中 `%` 匹配任意长度字符串，`_` 匹配单个字符。

设计思路

`LIKE` 是比较运算符的一种，应接入 `ComparisonExpr`，但它只对字符串类型有意义。因此求值前需要检查左右操作数类型。

实现链路

```
col LIKE 'a_%'
-> parser 生成 LIKE_OP
-> ComparisonExpr::compare_value
-> like_match
-> 返回布尔结果
```

关键代码锚点

- `src/observer/sql/parser/parse_defs.h`：比较操作枚举。
- `src/observer/sql/parser/yacc_sql.y`：解析 `LIKE`。
- `src/observer/sql/expr/expression.cpp`：比较表达式分发。
- `src/observer/sql/expr/like_match.*`：模式匹配实现。

测试验证

验证 `%`、`_`、普通字符混合匹配，非 `CHAR` 类型使用 `LIKE` 失败。

3.7 join-tables

功能目标

支持多表 `INNER JOIN ... ON ...`，包括链式 `JOIN`、`ON` 条件中的 `AND`、`JOIN` 与 `WHERE` 组合。

设计思路

`JOIN` 可以抽象为两个输入算子的组合。当前实现采用嵌套循环连接，先遍历左表，再遍历右表，对组合 `tuple` 应用 `ON` 谓词过滤。

实现链路

```
SELECT ... FROM t1 INNER JOIN t2 ON t1.id=t2.id
-> parser 收集 join_conditions
-> SelectStmt 绑定 JOIN 条件
-> JoinLogicalOperator
-> NestedLoopJoinPhysicalOperator
-> ON 谓词过滤组合 tuple
```

关键代码锚点

- `src/observer/sql/parser/yacc_sql.y`
- `src/observer/sql/stmt/select_stmt.cpp`
- `src/observer/sql/operator/join_logical_operator.h`
- `src/observer/sql/operator/nested_loop_join_physical_operator.*`

测试验证

验证两表 JOIN、多表 JOIN、JOIN ON + WHERE、非法字段报错。

3.8 function / expression

功能目标

支持表达式计算和标量函数，例如：

- `LENGTH(str)`
- `ROUND(float)`
- `DATE_FORMAT(date, format)`
- 算术表达式 `a + b * c`

设计思路

函数和算术都属于表达式系统。parser 构造未绑定函数表达式，binder 校验参数类型和数量，执行阶段按函数类型求值。

实现链路

```
SELECT length(name), round(score)
  -> FunctionExpr / ArithmeticExpr
  -> ExpressionBinder 校验参数
  -> ProjectPhysicalOperator 求值
```

关键代码锚点

- `src/observer/sql/expr/expression.*`
- `src/observer/sql/expr/sql_function.*`
- `src/observer/sql/parser/expression_binder.cpp`
- `src/observer/sql/parser/yacc_sql.y`

测试验证

验证合法函数返回值，参数个数和类型错误时语句失败。

3.9 order-by

功能目标

支持 `ORDER BY` 单列和多列排序，支持默认 ASC 和显式 DESC，可与 WHERE、JOIN、GROUP BY 联用。

设计思路

排序算子位于子查询结果之后、最终投影之前。当前采用内存排序方式：先收集子算子输出 tuple，再按 ORDER BY 表达式比较。

实现链路

```
SELECT ... FROM t WHERE ... ORDER BY c DESC
-> parser 收集 order_by
-> SelectStmt 绑定排序表达式
-> SortLogicalOperator
-> SortPhysicalOperator
-> Project 输出
```

关键代码锚点

- `src/observer/sql/parser/yacc_sql.y`
- `src/observer/sql/operator/sort_logical_operator.h`
- `src/observer/sql/operator/sort_physical_operator.*`

测试验证

验证单列、多列、ASC/DESC、与 WHERE/JOIN/GROUP BY 组合。

3.10 group-by

功能目标

支持多列 `GROUP BY`、分组聚合、`HAVING` 过滤，以及 WHERE、JOIN、ORDER BY 组合。

设计思路

GROUP BY 要在 WHERE 之后、Project 之前执行。HAVING 不能当作 WHERE 提前执行，因为 HAVING 过滤的是聚合后的分组结果。

实现链路

```
TableGet / Join
-> Predicate(WHERE)
-> HashGroupBy / ScalarGroupBy
-> Predicate(HAVING)
-> Sort(ORDER BY)
-> Project
```

关键代码锚点

- `src/observer/sql/parser/yacc_sql.y`：解析 `GROUP BY` 与 `HAVING`。
- `src/observer/sql/stmt/select_stmt.cpp`：绑定 group by 与 having 表达式。
- `src/observer/sql/optimizer/logical_plan_generator.cpp`：在 GroupBy 后插入 HAVING Predicate。
- `src/observer/sql/operator/hash_group_by_physical_operator.*`

测试验证

典型 SQL：

```
SELECT id, avg(score)
FROM t
GROUP BY id
HAVING avg(score) > 2
ORDER BY id DESC;
```

验证多字段分组、HAVING 聚合条件、HAVING 分组键条件、空表聚合。

3.11 multi-index

功能目标

支持复合索引：

```
CREATE INDEX idx ON t(c1, c2, ...);
```

并保证 INSERT、UPDATE、DELETE 后复合索引与堆表一致。

设计思路

单列索引只需要一个字段编码成 key，复合索引需要将多个字段按定义顺序拼接为组合 key，并按字典序比较。索引元数据也需要从单字段扩展为字段列表。

实现链路

```
CREATE INDEX idx ON t(a, b)
-> IndexMeta 保存字段列表
-> 建索引扫描已有记录，生成组合 key
INSERT / UPDATE / DELETE
-> 维护所有相关索引项
```

关键代码锚点

- `src/observer/sql/parser/yacc_sql.y`
- `src/observer/storage/index/index_meta.*`
- `src/observer/storage/index/bplus_tree_index.*`
- `src/observer/storage/table/heap_table_engine.*`

测试验证

验证空表建索引、非空表建索引、复合条件查询、DML 后索引一致性。

3.12 unique

功能目标

支持唯一索引：

```
CREATE UNIQUE INDEX idx ON t(c);
```

重复键插入或对已有重复数据建唯一索引应失败。

设计思路

唯一约束可放在索引层实现。插入索引项前先检查 key 是否存在，若存在则拒绝插入。索引元数据需要记录 unique 标志，并持久化。

实现链路

```
CREATE UNIQUE INDEX
    -> IndexMeta(unique=true)
INSERT / UPDATE
    -> index insert 前查重
    -> 重复则返回失败
```

关键代码锚点

- `src/observer/storage/index/index_meta.*`
- `src/observer/storage/index/bplus_tree_index.*`
- `src/observer/storage/table/heap_table_engine.*`

测试验证

验证重复值插入失败、重复数据上建唯一索引失败、不同值插入成功。

3.13 simple-sub-query

功能目标

支持简单子查询，包括：

```
expr IN (SELECT ...)
expr NOT IN (SELECT ...)
expr = (SELECT ...)
expr < (SELECT ...)
```

子查询输出必须是一列，多行标量子查询和 `SELECT *` 子查询应失败。

设计思路

旧 WHERE 条件链 `ConditionSqlNode -> FilterStmt` 只能表示简单二元比较，不能承载“需要独立执行的 SELECT”。因此 SELECT WHERE 被切换到表达式链，把子查询作为表达式节点接入。

核心思想：

子查询不是常量，而是能产生值或结果集的表达式。

实现链路

```
SELECT ... WHERE id IN (SELECT id FROM t2)
-> parser 构造 InSubQueryExpr + UnboundSubQueryExpr
-> ExpressionBinder 递归构造内部 SelectStmt
-> LogicalPlanGenerator 生成外层 Predicate
-> PhysicalPlanGenerator prepare 子查询物理计划
-> InSubQueryExpr materialize 内层结果并判断成员关系
```

关键代码锚点

- `src/observer/sql/parser/yacc_sql.y`: `select_where`、`where_predicate`、`select_subquery`。
- `src/observer/sql/stmt/select_stmt.cpp`: 绑定 `filter_exprs` 为 `where_expression_`。
- `src/observer/sql/parser/expression_binder.cpp`: `bind_subquery_expression`。
- `src/observer/sql/expr/subquery_expr.*`: `SubQueryExpr`、`InSubQueryExpr`、`prepare/open/close`。
- `src/observer/sql/optimizer/physical_plan_generator.cpp`: 谓词算子创建前 `prepare` 子查询。

核心伪代码

```
bind_subquery_expression(expr):
    stmt = SelectStmt::create(inner_select)
    assert stmt.output_columns == 1
    assert output is not STAR
    return SubQueryExpr(stmt, output_expr_copy)
```

测试验证

验证：

- `IN / NOT IN` 子查询。
- 标量聚合子查询比较。
- 子查询在比较左右两侧。
- 多行标量子查询失败。
- `SELECT *` 子查询失败。

3.14 null

功能目标

支持列定义 `NULL / NOT NULL`，支持插入 `NULL`、更新 `NULL`、`IS NULL / IS NOT NULL`，并保证普通比较和聚合函数符合 `NULL` 语义。

设计思路

`NULL` 不是普通值，不能用 `0` 或空字符串替代。实现上区分两类信息：

- `nullable`：schema 级约束，表示字段是否允许为空。
- `null bitmap`：record 级状态，表示某一行某一列当前是否为空。

record 布局调整为：

```
[系统事务字段][null bitmap][用户字段 payloads]
```

实现链路

```
CREATE TABLE t(a INT NOT NULL, b INT NULL)
-> AttrInfoSqlNode::nullable
-> FieldMeta::nullable_
-> TableMeta 计算 null_bitmap_offset_

INSERT INTO t VALUES (1, NULL)
-> Value::is_null()
-> Table::make_record 检查 NOT NULL
-> TableMeta::set_field_null

SELECT * FROM t WHERE b IS NULL
-> RowTuple::cell_at 读取 bitmap
-> Value::set_null(type)
-> ComparisonExpr::compare_value(IS_NULL)
```

关键代码锚点

- `src/observer/sql/parser/yacc_sql.y`: 解析 `NULL / NOT NULL / IS NULL`。
- `src/observer/common/value.*`: `Value::is_null_`。
- `src/observer/storage/field/field_meta.*`: `nullable_`。
- `src/observer/storage/table/table_meta.*`: `field_is_null / set_field_null`。
- `src/observer/storage/table/table.cpp`: 写记录时设置 bitmap。
- `src/observer/sql/expr/expression.cpp`: 普通比较遇 NULL 返回 false。
- `src/observer/sql/expr/aggregator.cpp`: 聚合跳过 NULL。

关键语义

```
b = NULL      -- 不命中
b IS NULL     -- 命中
```

普通比较遇到 NULL 按 unknown 处理，不直接视为 true；只有 `IS NULL` 和 `IS NOT NULL` 专门检查 NULL 状态。

测试验证

验证：

- NOT NULL 字段插入 NULL 失败。
- 可空字段插入 NULL 成功。
- `IS NULL / IS NOT NULL` 正确。
- 聚合函数跳过 NULL。
- NULL 与索引路径结果一致。

3.15 update-select

功能目标

支持：

```
UPDATE t SET c = (SELECT ...) WHERE ...
```

SET 右值可以是字面量、普通表达式或标量子查询。子查询可以是非关联，也可以引用当前更新行。

设计思路

UPDATE 不单独实现子查询，而是把 SET 右值升级为 `Expression`，复用 `simple-sub-query` 的表达式绑定与执行体系。

核心思想：

UPDATE 只负责“对每条命中行求 SET 表达式的值并写回”。
SET 右值如何求值，由 `Expression` 系统负责。

实现链路

```
UPDATE t1 SET val = (SELECT score FROM t2 WHERE t2.id=t1.id)
-> parser: update value_expr 为 expression
-> UpdateStmt::create: ExpressionBinder 绑定 value_expr
-> 子查询内部 SelectStmt 识别 outer_ref
-> PhysicalPlanGenerator prepare SET 子查询
-> UpdatePhysicalOperator 扫描命中旧记录
-> 每行调用 value_expr->get_value(row_tuple, value)
-> 写回字段，失败则 rollback_updates
```

关键代码锚点

- `src/observer/sql/parser/yacc_sql.y`: `UPDATE ID SET ID EQ expression where`。
- `src/observer/sql/stmt/update_stmt.cpp`: 绑定 SET 右值。
- `src/observer/sql/optimizer/physical_plan_generator.cpp`: prepare SET 右值中的子查询。
- `src/observer/sql/operator/update_physical_operator.cpp`: 逐行求值、写回、回滚。
- `src/observer/sql/expr/subquery_expr.cpp`: outer tuple 和相关子查询 materialize。

关键伪代码

```
for old_record in matched_records:
    row_tuple = RowTuple(old_record)
    value = value_expr.get_value(row_tuple)
    new_record = copy(old_record)
    set_update_field_value(new_record, value)
    trx.update_record(old_record, new_record)
```

先缓存旧记录再更新，是为了避免边扫描边修改导致漏扫或重复扫描；同时中途失败时可以按旧记录回滚。

测试验证

验证：

- SET 右值为字面量。
- SET 右值为聚合子查询。
- SET 右值为关联标量子查询。
- 多行标量子查询失败。
- 更新 NULL 时检查目标字段 nullable。
- 中途失败时已更新记录回滚。

3.16 complex-sub-query

功能目标

支持复杂子查询，包括：

- 嵌套子查询。
- 关联子查询，即内层 SELECT 引用外层当前行。
- 子查询中包含聚合。
- 非法多行标量子查询和 `SELECT *` 子查询失败。

设计思路

关联子查询的本质是作用域问题。内层 SELECT 在绑定字段时，如果字段不属于内层 FROM 表，就需要沿父作用域查找。如果找到外层字段，就把该字段标记为 outer reference。

执行时，关联子查询不能缓存全局结果，因为它依赖外层当前行；必须对外层每一行重新执行。

实现链路

```
SELECT * FROM t1
WHERE t1.id IN (
    SELECT t2.id FROM t2 WHERE t2.k = t1.k
)
```

绑定阶段：

```
t2.k -> 内层字段
t1.k -> 外层字段, FieldExpr::outer_ref_=true
SubQueryExpr::correlated_=true
```

执行阶段：

```
外层每一行 t1 tuple 入 outer tuple context
内层子查询执行时读取当前 t1.k
每行重新 materialize 子查询结果
```

关键代码锚点

- `src/observer/sql/parser/expression_binder.*`：父作用域链和外层字段绑定。
- `src/observer/sql/stmt/select_stmt.*`：子查询创建时传入父 binder context。
- `src/observer/sql/expr/expression.*`： `FieldExpr::outer_ref_`。
- `src/observer/sql/expr/subquery_expr.*`： `SubQueryOuterContext`、 `correlated_`、按行重新 materialize。

关键概念

非关联子查询：结果只依赖内层表，可缓存。

关联子查询：结果依赖外层当前行，不能缓存，必须逐行重算。

测试验证

验证：

- 嵌套非关联子查询。
- 一层关联子查询。
- 聚合 + 关联组合。
- 多行标量子查询失败。
- `SELECT *` 子查询失败。

3.17 big-write

功能目标

验证大量 INSERT、UPDATE、DELETE 混合操作下，系统能稳定运行，最终数据和索引一致，并支持重启后读取。

设计思路

big-write 不新增 SQL 语法，重点是 DML 路径的稳健性。尤其是更新和删除索引列时，需要保证堆表和 B+ 树同步。若某一步失败，应尽可能回滚已经完成的索引操作。

实现链路

大量随机写操作

```
-> INSERT / UPDATE / DELETE
-> Table / HeapTableEngine
-> 维护 record + all indexes
-> 最终 count(*) 校验
-> 重启后再次 count(*) 校验
```

关键代码锚点

- `src/observer/storage/table/heap_table_engine.*`
- `src/observer/storage/table/table.*`
- `src/observer/storage/index/*`
- `lab/test/stress/*`

测试验证

压力脚本使用固定随机种子，覆盖数百到数千次写操作，最终校验行数和重启后持久化结果。

3.18 big-query

功能目标

验证较大数据量下 SELECT、索引过滤、JOIN、GROUP BY、HAVING、IN 子查询、ORDER BY 等查询路径组合后仍然正确稳定。

设计思路

big-query 是压力验证题，不是新增语法题。设计重点是构造可复现、覆盖面足够、不会产生巨大笛卡尔积的压力 SQL。

压力生成器特点：

- 主表 `bq_main` 800 行。
- 维表 `bq_dim` 20 行。
- 固定随机种子 `SEED=7`。
- 最后一条固定为 `SELECT count(*) FROM bq_main;`，期望 800。

实现链路

```
gen_big_query_sql.py
-> CREATE TABLE / CREATE INDEX
-> INSERT 800 + 20 行
-> 点查、索引 count、group by、having
-> IN 子查询 + order by
-> 二表 join count
-> 最终 count(*) = 800
```

关键代码锚点

- `lab/test/stress/gen_big_query_sql.py`
- `lab/test/manifest.json` 中 `stress-big-query`
- `lab/test/run_all.py` stress runner

测试验证

运行：

```
python3 lab/test/run_all.py --only stress-big-query
```

验证最终 `count(*)` 为 800，且中间查询无失败。

4. 测试与验证

4.1 测试体系

本组测试覆盖官方用例、自定义 SQL、综合断言与压力测试。测试脚本会启动 observer，执行 SQL，统计 SUCCESS/FAILURE，并对部分查询结果做精确比对。

测试来源：

- 官方 primary 用例。
- 自定义 SQL 套件。
- 综合断言用例。
- big-write / big-query 压力脚本。

4.2 全量测试结果

测试结果来源：`lab/test/latest.md`。

指标	数值
开始时间	2026-06-03T16:33:59+08:00
结束时间	2026-06-03T16:35:50+08:00
分支	1357
commit	bf31e7c6
observer	/Users/yishuoyan/projects/bupt/25-26-2/miniob-sql-task-2/build_debug/bin/observer
用例总数	135
通过	135
失败	0
跳过	0
通过率	100.0%
总耗时	111.5s

截图占位：全量测试报告汇总截图，应展示 135/135 通过、失败 0、通过率 100%。

4.3 官方用例覆盖

官方用例覆盖以下功能：

- basic
- drop-table
- update
- date
- aggregation-func
- join-tables
- order-by
- group-by
- multi-index
- function / expression
- unique
- simple-sub-query
- complex-sub-query
- null

所有官方用例均通过。

4.4 自定义与压力用例

自定义 SQL 覆盖：

- update-index
- date-delete
- like
- join
- group-by
- unique
- simple-sub-query
- complex-sub-query
- null
- update-select

压力用例覆盖：

- big-write
- big-query
- 重启持久化检查

4.5 代表性验证 SQL

NULL

```
CREATE TABLE t(id INT NOT NULL, score INT NULL);
INSERT INTO t VALUES (1, NULL);
SELECT * FROM t WHERE score IS NULL;
```

simple-sub-query

```
SELECT id FROM ssq_1
WHERE id IN (SELECT id FROM ssq_2);
```

complex-sub-query

```
SELECT * FROM t1
WHERE id IN (
    SELECT id FROM t2 WHERE t2.k = t1.k
);
```

update-select

```
UPDATE us_t1
SET val = (SELECT score FROM us_t2 WHERE us_t2.id = us_t1.id);
```

big-query

```
SELECT id FROM bq_main
WHERE k IN (SELECT k FROM bq_dim WHERE label > 500)
ORDER BY id;
```

截图占位：以上代表性 SQL 的 observer CLI 执行截图。

5. 总结与心得

5.1 工作总结

本实验在 MiniOB 框架上完成了 18 项功能，覆盖 SQL 解析、语义绑定、表达式系统、查询计划、物理算子、事务调用、表存储和 B+ 树索引维护等多个层次。所有功能均沿 MiniOB 原有主链实现，没有通过旁路脚本直接绕过数据库内核。

从功能上看，系统已经支持基础 DDL/DML、DATE、NULL、聚合、LIKE、JOIN、函数、ORDER BY、GROUP BY、复合索引、唯一索引、简单和复杂子查询、UPDATE 子查询，以及大规模读写压力验证。

从架构上看，几个关键模块形成了复用关系：

- 表达式系统支撑函数、比较、NULL、子查询和 UPDATE SET。
- 子查询表达式同时服务于 SELECT WHERE、complex-sub-query 和 update-select。
- 索引维护逻辑同时影响 update、multi-index、unique、big-write。
- 压力测试用于验证多个功能组合后的稳定性。

5.2 技术收获

本次课程设计加深了对数据库系统分层结构的理解：

- parser 只负责语法结构，不应承担过多执行语义。
- binder 是 SQL 名称解析和作用域处理的关键，尤其在关联子查询中非常重要。
- 逻辑计划描述“做什么”，物理算子描述“怎么做”。
- 火山模型通过 `open / next / close` 接口统一串联扫描、过滤、连接、聚合、排序和投影。
- 索引与堆表必须保持一致，DML 中的失败回滚是工程正确性的关键。
- NULL 与子查询看似是 SQL 语法题，实际会影响存储、表达式、聚合、执行器多个层次。

5.3 不足与改进

当前系统仍存在以下不足：

- 未实现 TEXT、视图、别名、create-table-select、update-mvcc 等扩展题。
- 排序和部分聚合采用内存实现，面对更大数据量时需要外部排序或更细致的内存控制。
- JOIN 以嵌套循环为主，缺少成本优化和 hash join。
- 子查询虽然支持关联场景，但仍可继续优化缓存策略和执行计划复用。
- 当前报告截图仍需在最终 Word 版中补充真实运行界面。

5.4 结论

本系统完成了课程设计要求中的 18 项功能，并通过 `lab/test/latest.md` 记录的全量测试：135 个用例全部通过，失败 0，跳过 0，通过率 100.0%。在当前测试覆盖范围内，系统功能符合预期，能够支撑课程设计提交与答辩演示。

附录 A：常用编译与运行命令

```
# 初始化依赖
bash build.sh init

# 编译 debug 版本
bash build.sh debug --make -j8

# 启动 observer CLI
build_debug/bin/observer -f etc/observer.ini -P cli

# 运行全量测试
python3 lab/test/run_all.py

# 只运行 big-query 压力测试
python3 lab/test/run_all.py --only stress-big-query
```

附录 B：截图清单

最终 Word 版建议插入以下截图：

截图	建议位置
macOS 或 Docker 编译成功	第 2 章环境配置
observer CLI 启动并 <code>show tables</code>	第 2 章环境配置
null 代表 SQL 执行结果	第 3.14 节
simple-sub-query 代表 SQL 执行结果	第 3.13 节
complex-sub-query 代表 SQL 执行结果	第 3.16 节
update-select 代表 SQL 执行结果	第 3.15 节
big-query 压力测试结果	第 3.18 节或第 4 章
全量测试 135/135 通过	第 4.2 节

附录 C：参考资料

- MiniOB 官方项目与课程指导书。
- `lab/题目/指导书.md`

- `lab/test/latest.md`
- `lab/log/yys-dev/` 功能开发日志。
- `lab/log/xc/` 功能开发日志与展示说明。